

第51讲 核心概念速查卡

RNN vs Transformer: 只保留最需要长期记忆的结构关系

概念	最短记忆
RNN	state_t 依赖 state_{t-1}, 时间轴递推
LSTM/GRU	改善递推中的记忆/梯度, 但仍保留时间串行依赖
Self-Attention	一个 Token 可以在一层中直接读取其他允许位置
路径长度	RNN 长程路径随距离增长; 单层 Attention 可约为1
训练并行	Transformer 的 Q/K/V 与 score matrix 可矩阵化并行
Causal Mask	未来位置 score 置为 $-\infty$; 训练仍可一次算所有位置
生成	自回归新 Token 尚未知, 所以生成仍逐 Token
KV Cache	缓存历史 K/V, 避免生成时重复计算
标准 Attention 代价	score matrix 为 (T,T), pair 数量 T^2

三个关键公式

RNN: $h_t = f(x_t, h_{t-1})$

Attention(Q,K,V) = softmax(QK^T / sqrt(d_k)) V

(T,d_k) @ (d_k,T) → (T,T) □ 标准 score 数量 T^2

科研判断

- 不要把 FLOPs、sequential depth、memory、throughput、latency 混成同一个“效率”。
- “训练并行”不等于“自回归生成并行”。
- “上下文窗口更长”不等于“真正利用长程信息更好”。
- RNN/LSTM/GRU 仍可能适合流式、固定状态、小设备等场景。

一眼比较表

维度	RNN/LSTM/GRU	Transformer Self-Attention
Token 交互	通过递推 state 间接传递	动态直接聚合其他 Token
长程路径	随距离增长	单层直接路径
时间轴并行	受 $state_t \leftarrow state_{t-1}$ 限制	训练层内高度可并行
标准长序列代价	时间递推；无 $T \times T$ score	标准 Attention 有 T^2 score
流式处理	自然	需要 KV Cache / 特殊机制
LLM 主流	历史重要	当前大模型核心

口述自检

- 为什么 LSTM 缓解梯度问题，却没有解决时间轴串行问题？
- 为什么 Causal Attention 可以训练并行，但生成仍然串行？
- 为什么 Transformer 的长程路径短，却仍然存在长上下文成本？
- 标准 Attention 的 T^2 到底来自哪个 Tensor Shape？